IME 775 — Lecture 20

3D Convolution, Transposed Convolution, and Pooling

---

1. Three-Dimensional Convolution

1.1 Why 3D Convolution?

A video is a sequence of frames forming a **spatio-temporal (ST) volume** — a 3D entity with dimensions height (H), width (W), and time (T). Analyzing one frame at a time via 2D convolution misses **motion information**, which can only be captured by examining multiple successive frames together.

Example: A single image of a half-open door cannot tell us whether the door is opening or closing. We need multiple successive frames to determine the direction of motion.

1.2 Graphical View

Imagine a brick (the **3D kernel**) sliding through the entire volume of a room (the **input ST volume**):

1. The brick slides left-to-right across the width
2. Then drops down and repeats across the height
3. Then advances along the time axis and repeats
4. At each slide stop: multiply element-wise and sum → one output voxel
5. The output is a 3D ST volume

1.3 The 3D Convolution Equation

$$Y_{t,y,x} = \sum_{k=0}^{k_T-1} \sum_{i=0}^{k_H-1} \sum_{j=0}^{k_W-1} X_{t+k,\, y+i,\, x+j}\, W_{k,i,j} \qquad \forall\, (t,y,x) \in S_o$$

1.4 Video Motion Detection

A moving object changes pixel values at the same spatial location across successive frames. A $2 \times 3 \times 3$ kernel can detect motion by computing the **temporal difference of spatial averages**:

$$W_{t=0} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & -1 & -1 \\ -1 & -1 & -1 \end{bmatrix}, \qquad W_{t=1} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The output is **high in regions of motion** (where the spatial averages differ between frames) and **low in static regions**.

Workout: A $2 \times 2 \times 2$ motion detection kernel with weights $W_{t=0} = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$ and $W_{t=1} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ is applied to a video. At a particular slide stop, the 2×2 region in frame t has all values 100 and in frame $t+1$ has all values 50. What is the output?

Solution:

$$Y = (-1)(100) \times 4 + (1)(50) \times 4 = -400 + 200 = -200$$

The large magnitude indicates motion at this location.

---

2. PyTorch: 3D Convolution

PyTorch expects **5D tensors** for 3D convolution:

Tensor	Shape	Description
Input	$N \times C \times D \times H \times W$	Batch × Channels × Depth (time) × Height × Width
Kernel	$C_{\text{out}} \times C_{\text{in}} \times k_T \times k_H \times k_W$	Output channels × Input channels × temporal × spatial

kernel

```
import torch
import torch.nn as nn

w_2d_smoothing = torch.tensor([
    [1.0, 1.0, 1.0],
    [1.0, 1.0, 1.0],
    [1.0, 1.0, 1.0],
]).unsqueeze(0)  # [1, 3, 3]

# Stack negative and positive versions for temporal differencing
w_3d = torch.cat([-w_2d_smoothing, w_2d_smoothing], dim=0)  # [2,
3, 3]

# Reshape to Conv3d weight format: [C_out, C_in, kT, kH, kW]
w_3d = w_3d.unsqueeze(0).unsqueeze(0)  # [1, 1, 2, 3, 3]

conv3d = nn.Conv3d(1, 1, kernel_size=(2, 3, 3), stride=1,
padding=0, bias=False)
conv3d.weight = nn.Parameter(w_3d, requires_grad=False)

# Example: 5 grayscale frames of 8x8
x = torch.randn(1, 1, 5, 8, 8)
with torch.no_grad():
    y = conv3d(x)
print(f"Input shape:  {x.shape}")      # [1, 1, 5, 8, 8]
print(f"Output shape: {y.shape}")       # [1, 1, 4, 6, 6]
```

---

3. Transposed Convolution (Fractionally Strided Convolution)

3.1 Motivation

In a standard convolution $\vec{y} = W\vec{x}$, we go from a larger input to a smaller output (downsampling). **Transposed convolution** multiplies by W^T, going from a smaller input to a larger output (upsampling).

3.2 The Matrix View

For a 1D convolution with kernel $\vec{w} = [w_0, w_1, w_2]$, input size 5, stride 1, valid padding:

$$W = \begin{bmatrix} w_0 & w_1 & w_2 & 0 & 0 \\ 0 & w_0 & w_1 & w_2 & 0 \\ 0 & 0 & w_0 & w_1 & w_2 \end{bmatrix}, \qquad \vec{y} = W\vec{x} \quad (3 \times 1)$$

The transposed operation:

$$\tilde{x} = W^T\vec{y} = \begin{bmatrix} w_0 & 0 & 0 \\ w_1 & w_0 & 0 \\ w_2 & w_1 & w_0 \\ 0 & w_2 & w_1 \\ 0 & 0 & w_2 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} \quad (5 \times 1)$$

3.3 Key Observations

- We **cannot** perfectly recover $\vec{x}$ from $\vec{y}$ — information was lost during the forward convolution (the weight matrix is non-square and non-invertible)
- Transposed convolution **distributes** output elements back in the same proportions as the forward convolution collected them — analogous to backpropagation's blame distribution
- It generates an output of the same size as the original input

3.4 Output Size

$$o' = (n' - 1) \cdot s + k - 2p$$

where n' is the input size to the transposed convolution.

Workout: Transposed convolution with stride $s = 2$, kernel size $k = 2$, valid padding ($p = 0$), input size $n' = 2$. What is the output size?

Solution:

$$o' = (2 - 1) \times 2 + 2 - 0 = 2 + 2 = 4$$

---

4. Application: Autoencoders and Embeddings

4.1 The Encoder-Decoder Architecture

An **autoencoder** consists of:

1. **Encoder:** Maps input → embedding (smaller representation) via convolution layers (downsampling)
2. **Decoder:** Reconstructs input from embedding via **transposed convolution** layers (upsampling)

The loss is the difference between original input and reconstructed output. Training minimizes this reconstruction error.

4.2 Why Transposed Convolution?

The encoder uses convolution to progressively reduce spatial dimensions. The decoder needs to **increase** spatial dimensions back to the original size. Transposed convolution learns the optimal upsampling function during training, rather than using predefined interpolation (nearest neighbor, bilinear, etc.).

4.3 Upsampling via Transposed Convolution in PyTorch

```
import torch
import torch.nn as nn

x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
w = torch.tensor([[5.0, 6.0], [7.0, 8.0]])

x = x.unsqueeze(0).unsqueeze(0)  # [1, 1, 2, 2]
w = w.unsqueeze(0).unsqueeze(0)  # [1, 1, 2, 2]  (C_in x C_out x
kH x kW)

transpose_conv = nn.ConvTranspose2d(1, 1, kernel_size=2, stride=2,
bias=False)
transpose_conv.weight = nn.Parameter(w, requires_grad=False)

with torch.no_grad():
    y = transpose_conv(x)
print(f"Input shape:  {x.shape}")   # [1, 1, 2, 2]
print(f"Output shape: {y.shape}")   # [1, 1, 4, 4]
```

```
print("Upsampled output:\n", y.squeeze())
```

---

5. Adding Convolution Layers to a Neural Network

In practice, we do **not** set kernel weights manually. We specify the architecture (kernel size, stride, padding, number of channels) and let the weights be **learned** through backpropagation.

```
import torch
import torch.nn as nn

class SampleCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=16,
                      kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.Conv2d(in_channels=16, out_channels=32,
                      kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.Conv2d(in_channels=32, out_channels=64,
                      kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
        )
        self.classifier = nn.Linear(64 * 14 * 14, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

model = SampleCNN(num_classes=10)
x = torch.randn(1, 1, 28, 28)
print(f"Output shape: {model(x).shape}")  # [1, 10]
```

6. Pooling

6.1 Why Pooling?

Convolution layers are sensitive to the exact location of features. Minor shifts in input (due to camera angle, rotation, cropping) can produce different feature maps. **Pooling** performs downsampling to achieve **local translation invariance** — a lower-resolution feature map still contains important features but is robust to small spatial shifts.

6.2 Max Pooling

At each kernel position, output the **maximum** value from the local patch.

Effect: Retains the strongest feature activation in each neighborhood.

6.3 Average Pooling

At each kernel position, output the **average** value of the local patch.

Effect: Smooths activations within each neighborhood.

6.4 Pooling Parameters

A 2×2 kernel with stride 2 reduces each spatial dimension by half (output is $\frac{1}{4}$ of input size). A 3×3 kernel with stride 3 reduces each dimension by one-third.

Workout: A 6×6 feature map is max-pooled with a 2×2 kernel and stride 2. What is the output size?

Solution:

$$o_H = \frac{6}{2} = 3, \qquad o_W = \frac{6}{2} = 3$$

Output size: 3×3.

6.5 PyTorch Implementation

```
import torch
import torch.nn as nn

x = torch.tensor([
    [31.0, 43.0, 57.0, 70.0],
    [25.0, 38.0, 50.0, 63.0],
    [19.0, 31.0, 44.0, 57.0],
    [12.0, 26.0, 39.0, 51.0],
]).unsqueeze(0).unsqueeze(0)  # [1, 1, 4, 4]

max_pool = nn.MaxPool2d(kernel_size=2, stride=2)
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

print("Max pooling:\n", max_pool(x).squeeze())
# [[43, 70],
#  [31, 57]]

print("Avg pooling:\n", avg_pool(x).squeeze())
# [[34.25, 60.00],
#  [22.00, 47.75]]
```

---

7. Putting It All Together: A Typical CNN Pipeline

A convolutional neural network typically chains these operations:

$$\text{Input} \to \underbrace{[\text{Conv} \to \text{ReLU} \to \text{Pool}]}_{\text{repeat } L \text{ times}} \to \text{Flatten} \to \text{FC} \to \text{Softmax}$$

Stage	Operation	Purpose
Feature extraction	Conv + ReLU	Detect local patterns at increasing complexity
Downsampling	Pooling	Reduce spatial size, gain translation invariance

Classification	FC + Softmax	Map flattened features to class probabilities

The kernel weights in convolution layers are **learned** via backpropagation — the network automatically discovers what local features are useful for the task.

Key Takeaways

1. **3D convolution** slides a brick through an ST volume — essential for **video analysis** and **motion detection**
2. **Transposed convolution** multiplies by W^T to **upsample** — used in decoders/autoencoders to reconstruct images from embeddings
3. The transposed operation cannot perfectly invert convolution (information is irretrievably lost during downsampling)
4. **Pooling** (max or average) downsamples feature maps for **local translation invariance** and reduced computation
5. A typical CNN pipeline: $\text{Conv} \to \text{ReLU} \to \text{Pool}$ (repeated), then $\text{FC} \to \text{Softmax}$
6. In real networks, convolution weights are **learned** through training, not manually specified
7. PyTorch uses `Conv3d` for 3D convolution, `ConvTranspose2d` for transposed 2D convolution, and `MaxPool2d` / `AvgPool2d` for pooling